Министерство образования Российской Федерации

ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ УПРАВЛЕНИЯ

Факультет: Прикладная информатика
Кафедра: Информационные технологии и системная аналитика

ЯЗЫКИ И МЕТОДЫ ПРОГРАММИРОВАНИЯ

Лабораторное занятие №3 на тему:
«Функции и рекурсия»

Преподаватель:
Авдеев В.С.

Студент:
Маслов Глеб
Алексеевич

Группа:
-

Москва 2026

Цель занятия

- Научиться создавать функции
- Работать с массивами в функциях
- Использовать рекурсию функций

Ход занятия

Функция — это набор инструкций, который может быть вызван по имени. Функции могут принимать параметры (аргументы) и возвращать результат. В Java функции называются методами и объявляются внутри классов (Рис. 1).

```
public static void имя()
{
    код метода
}
```

Рис. 1 - Структура функции

Для создания функции в Java, следует выполнить следующие шаги:

1. Определить имя функции
2. Определить тип возвращаемого значения
3. Определить параметры (если есть)
4. Написать тело функции

Давайте создадим простую функцию, которая принимает два числа и возвращает их сумму (Рис. 2):

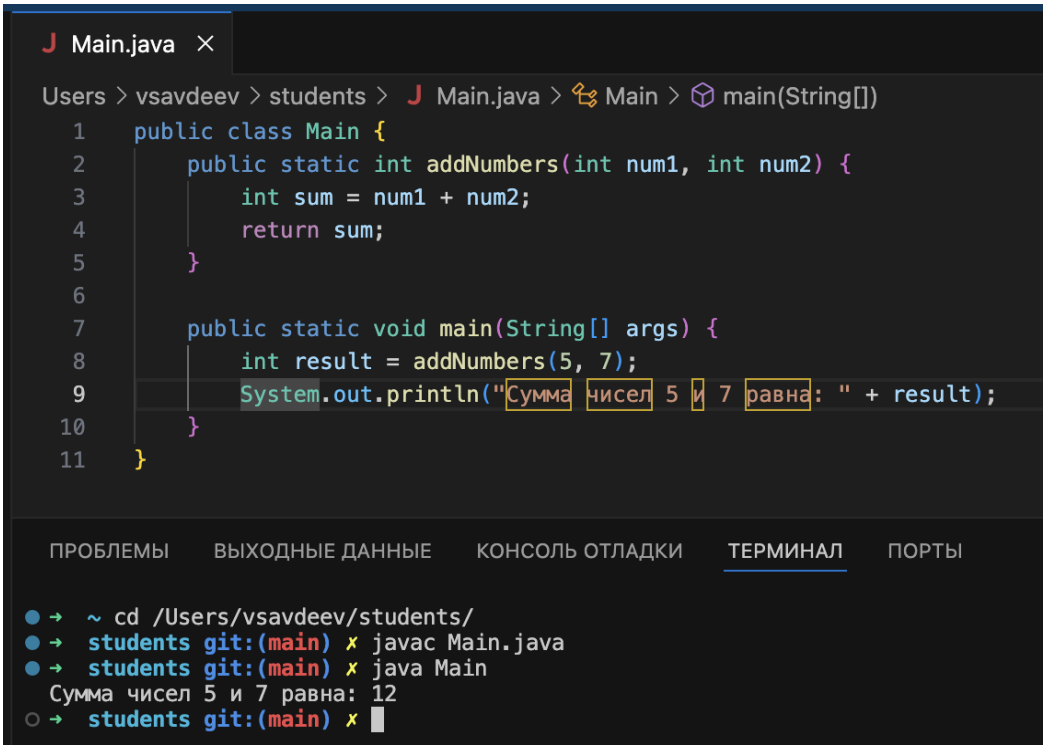


Рис. 2 - Функция на языке Java

Мы создали функцию addNumbers, которая принимает два параметра типа int и возвращает результат типа int. В main методе мы вызываем функцию addNumbers, передаем ей числа 5 и 7, и выводим результат на экран.

В Java выделяют 2 вида функций:

- Функция, которая что-либо возвращает.
- Функция, которая не возвращает ничего.

Поскольку мы хотим обращаться к функциям напрямую без создания объекта, то мы прописываем ключевое слово static. Оно говорит компилятору что функция принадлежит всему классу, а не конкретным объектам.

Для создания функций необходимо указать возвращаемый тип данных, указать название и параметры. В случае когда функция ничего не возвращает, то указывается тип данных void.

Правила написания кода:

- Названия функций должны быть говорящими и отражать их назначение.
- Старайтесь делать функции небольшими и выполняющими одну конкретную задачу.
- Используйте параметры для передачи данных в функцию и возвращаемые значения для получения результатов.
- Помните, что функции в Java должны быть объявлены внутри классов.

Воспользуйтесь git и запушьте ваши наработки в новый репозиторий (проект) на GitFlic с названием lab3

Чтобы передать массив в метод Java, нужно в качестве аргумента указать имя массива, который нужно передать. Например:

```
public void myMethod(int[] myArray) {
    // some code here
}
```

Массив в Java (Java Array) — это структура данных, которая хранит набор пронумерованных значений одного типа (элементы массива).

В данном примере myMethod принимает аргумент myArray типа int[]. Чтобы вызвать этот метод и передать в него массив, можно сделать так:

```
int[] myArray = {1, 2, 3, 4, 5};
myMethod(myArray);
```

В этом примере мы создаем массив myArray и инициализируем его некоторыми значениями, а затем передаем его в метод myMethod вызовом myMethod(myArray).

Обратите внимание: в Java элементы массива нумеруются с нуля. То есть, если у нас есть массив на 10 элементов, то первый элемент массива будет иметь индекс 0, а последний — 9 (Рис 3.).

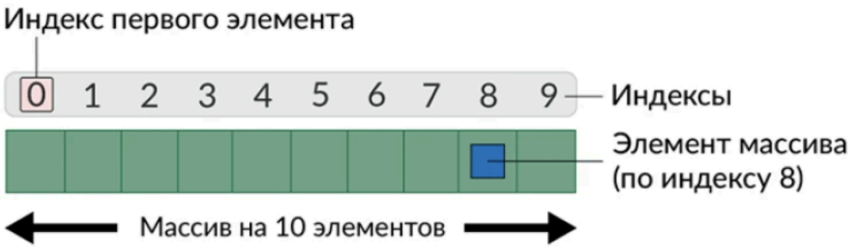


Рис. 3 - Массив

Давайте доработаем нашу функцию, чтобы она принимала массив значений и возвращала их сумму (Рис. 4):

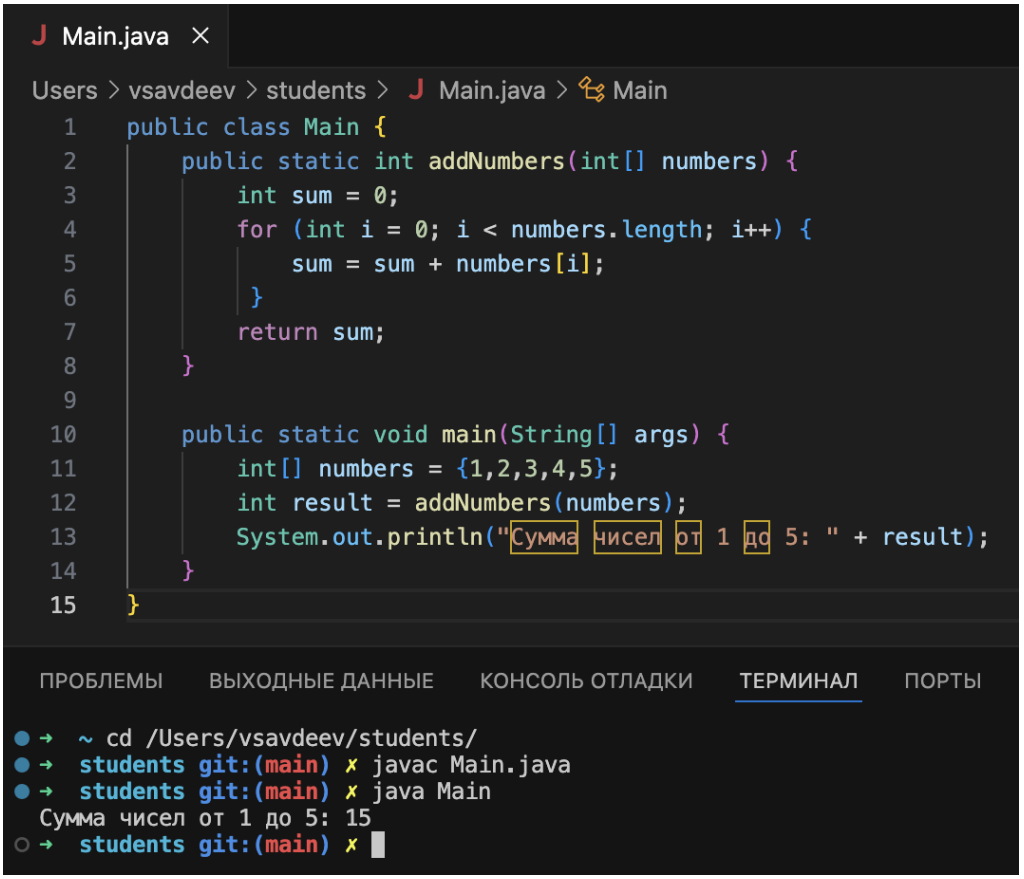


Рис. 4 - Функция с массивом

Воспользуйтесь git и запушьте ваши доработки в ранее созданный репозиторий (проект) на GitFlic с названием lab3

Рекурсия - это средство, которое позволяет методу вызывать самого себя. Такой метод называется рекурсивным.

Когда рекурсивный метод вызывает самого себя, новым локальным переменным и параметрам выделяется место в стеке и код метода выполняется с этими новыми исходными значениями. При каждом возврате из вызова рекурсивного метода прежние локальные переменные и параметры удаляются из стека, а выполнение продолжается с точки вызова в самом методе.

Давайте напишем функцию вычисления факториала, определяемая для положительных целых чисел n формулой
(Рис. 5):

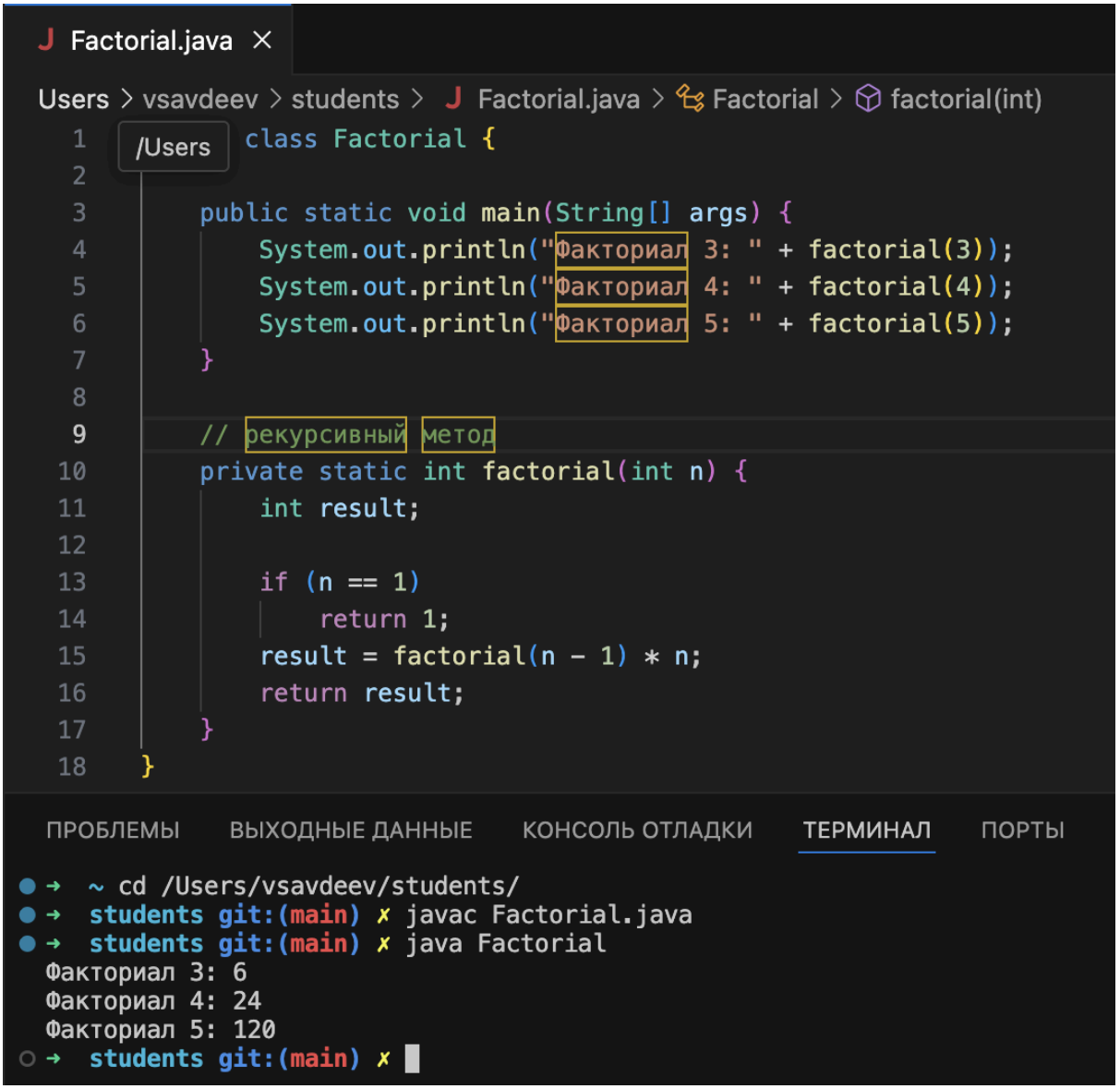


Рис. 5 - Факториал

В нашей реализации f a c t o r i a l ( ) можно выделить два компонента, присутствующих в каждой рекурсивной функции. Во-первых, базовый случай возвращает значение без последующих рекурсивных вызовов. Он делает это для одного или для нескольких специальных входных значений, для которых функция вычисляется без рекурсии. Для fa c to r ia l () базовым случаем является n = 1. Во-вторых, центральное место в рекурсивной функции занимает свертка: результат функции для одного (или нескольких) аргументов связывается с результатом этой же функции, вычисленным для другого (или нескольких других) аргумента. Для fa c to r ia l () свертка определяется формулой n * fa c to r ia l(n - l) . Эти два компонента должны присутствовать во всех рекурсивных функциях. Кроме того, последовательность значений аргументов должна сходиться к базовому случаю. Для fa c to r ia l() значение n уменьшается на 1 с каждым вызовом, поэтому последовательность значений аргументов сходится к базовому случаю n = 1.

Воспользуйтесь git и запушьте ваши доработки в ранее созданный репозиторий (проект) на GitFlic с названием lab3

Далее решим задачи

№ задачи	Условие	Комментарий
1	Дан массив целых чисел и ещё одно целое число. Удалите все вхождения этого числа из массива (пропусков быть не должно).	
2	Напишите рекурсивную функцию, которая получает аргумент — целое число n и возвращает ln (n!).	
3	Дано натуральное число n>1. Проверьте, является ли оно простым. Программа должна вывести слово YES, если число простое и NO, если число составное.	Алгоритм должен иметь сложность O(logn).
4	Дано слово, состоящее только из строчных латинских букв. Проверьте, является ли это слово палиндромом. Выведите YES или NO.	При решении этой задачи нельзя пользоваться циклами
5	Функция Аккермана A(m,n), играет важную роль в теории вычислимости. Она определена следующим образом: $$A(m,\ n) = \begin{cases} n+1, & m = 0; \\ A(m-1,\ 1), & m > 0,\ n = 0; \\ A(m-1,\ A(m,\ n-1)), & m > 0,\ n > 0. \end{cases}$$ Условие задачи: есть 2 целых неотрицательных числа m и n, каждое из них находится в отдельной строке. Необходимо вывести A(m,n).	